Ders 7 Genişletilmiş Notu: Veri Yapıları, Prefix-Infix-Postfix Gösterimleri, Algoritma Analizi ve Asimptotik Gösterimler

Ders 7 Genişletilmiş Notu: Veri Yapıları, Prefix-Infix-Postfix Gösterimleri, Algoritma Analizi ve Asimptotik Gösterimler

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Algoritmalar Bölümünün Devamı ve Veri Yapıları
- 2. Bağlı Listeler
- 3. Kuyruk
- 4. Yığın
- 5. Infix, Prefix ve Postfix Gösterimleri
- 6. İfadeyi İkili Ağaca Yerleştirme ve Dönüşüm Örnekleri
- 7. Sayısal Bir Örnek ve Üs Alma
- 8. Yığın ile İfade Değerlendirme
- 9. Gösterimler Arası Dönüşümler
- 10. Algoritma Analizine Giriş
- 11. Algoritma Analizinin Temel Kavramları
- 12. Yürütme Zamanı Hesabı: Ortalama ve En Küçük Eleman
- 13. Yürütme Zamanı Hesabı: Seçmeli Sıralama
- 14. Yürütme Zamanı Hesabı: Matris Toplama
- 15. Fonksiyonların Büyümesi
- 16. En Kötü, En İyi ve Ortalama Durum
- 17. Algoritma Seçimi: Büyüme Hızına Göre Karar
- 18. Asimptotik Gösterim ve Karmaşıklık Sınıfları
- 19. Big-O Tanımı ve İspat Örneği
- 20. Sıralı Arama ve Bilinen Algoritmaların Karmaşıklığı
- 21. Big-Omega ve Big-Theta
- 22. Limit ve Tümevarım ile Karmaşıklık
- 23. Birden Fazla Parçadan Oluşan Fonksiyonların Karmaşıklığı
- 24. Yinelemeli Algoritmaların Analizi
- 25. Dersin Kapanışı
- Örnek Sorular
- Dersin Toparlaması

1. Algoritmalar Bölümünün Devamı ve Veri Yapıları

Ders Zoom üzerinden yapıldı. Kaydın ilk dakikaları duyurular ve Zoom'daki bir ad karışıklığıyla geçer; asıl anlatım yaklaşık 00:11:40'ta, geçen hafta kalınan yerden başlar. Bu ders, önceki hafta başlanan algoritmalar bölümünün devamıdır.

Hoca başta bir hatırlatma yaptı: Bu bölümde görülen her şey, ileride Veri Yapıları ve Algoritma Analizi derslerinde çok daha ayrıntılı olarak görülecek. Ayrık Matematik dersinin amacı, bilgisayar bilimlerindeki derslerin hepsine az çok dokunabilmektir. Hoca, bu derste görülenlerin

en az yüzde sekseninin, başka bir derste ya da algoritma kurarken ve kod yazarken farkında olmadan kullanılacağını söyledi.

> **Hocanın vurgusu:** Bu dersin içinde gördüğünüz şeyleri ilerleyen sınıflarınızda mutlaka, farkında olmadan kullanacaksınız.

1.1 Veri Yapısı Nedir?

"Veri yapıları deyince aklınıza ne gelir?" sorusunun cevabı geniştir: kümeler, ağaçlar, graflar, kuyruklar (queue), yığınlar (stack), bağlı listeler (linked list), diziler (array), matrisler ve hash tabloları birer veri yapısıdır. Veri yapısı, üzerinde veriyi tuttuğumuz ve verinin özelliklerine göre hareket ettiğimiz yapıdır.

1.2 Diziler

Dizinin nasıl çalıştığını hepimiz biliyoruz. Dizi, bellekte (memory) ardışık olarak sıralanmış gözlerden oluşur; örneğin sıfırıncı indisten sekizinci indise kadar. Bir dizinin elemanlarına sırayla, teker teker erişilebilir. Rastgele (random) erişim de mümkündür: herhangi bir göze doğrudan gidilebilir.

2. Bağlı Listeler

Dizilerden türetilmiş bir yapı da bağlı listedir (linked list). Hoca bunu bir örnekle anlattı.

Dizinin elemanları $5, 7, 1, 8, 4$ olsun ve indisleri $1, 2, 3, 4, 5$ olsun. Bu diziyi küçükten büyüğe sıralayacağız, ama elemanların yerini değiştirmeden, bağlı liste mantığıyla. Her gözün yanına bir **link adresi** yazılır: bu gözdeki veri okunduktan sonra hangi göze gidilmesi gerektiğini gösterir.

- En küçük eleman 1'dir ve 3 numaralı gözdedir. Bu yüzden başlangıç (start) noktası 3 numaralı gözdür.
- 1'den sonra gelen eleman 4'tür; 5 numaralı gözdedir. Yani 3 numaralı gözün linki 5'tir.
- 4'ten sonra 5 gelir; 1 numaralı gözdedir. 5 numaralı gözün linki 1'dir.
- 5'ten sonra 7 gelir; 2 numaralı gözdedir. 1 numaralı gözün linki 2'dir.
- 7'den sonra 8 gelir; 4 numaralı gözdedir. 2 numaralı gözün linki 4'tür.
- 8 son elemandır; 4 numaralı göz "dur" (stop) gözüdür.

İndis	Veri	Link
1	5	2
2	7	4
3	1	5
4	8	dur
5	4	1

Gezinti başlangıçtan yapılır: 3. göz (1) → 5. göz (4) → 1. göz (5) → 2. göz (7) → 4. göz (8), dur. Böylece dizi, link adresleri üzerinden sıralanmış olur: $1, 4, 5, 7, 8$.

2.1 Bağlı Listeye Eleman Ekleme

Hoca, Veri Yapıları dersinde "bağlı listeden bir eleman çıkar" ya da "yeni eleman ekle" gibi işlerin isteneceğini söyledi. Eleman çıkarıldığında ya da eklendiğinde linkler değişir; bu değişikliği yapan algoritmayı yazmanız beklenecek.

Örnek olarak 6 eklenir:

- Başlangıçtan gelinir: 6, 1'den büyük mü? Evet. Linke göre 5. göze gidilir.
- 5. gözde 4 var. 6, 4'ten büyük mü? Evet. 1. göze gidilir.
- 1. gözde 5 var. 6, 5'ten büyük mü? Evet. 2. göze gidilir.
- 2. gözde 7 var. 6, 7'den büyük mü? Hayır. O hâlde 6, 5 ile 7 arasına girmelidir.
- 6, yeni bir göze, 6 numaralı göze yazılır.

- Buraya gelinmesini sağlayan link, yani 1. gözün (veri 5) linki artık 2 değil 6 olur.
- 6 numaralı gözün linkine, eski link olan 2 yazılır. Böylece 6'dan sonra 2. göze (7) gidilir ve gezinti devam eder.

Silme işleminde de algoritmada buna benzer bir değişiklik yapılır. Hoca burada algoritmaların ayrıntısına girmediklerini, yalnız "bağlı liste" dendiğinde neyin anlaşılması gerektiğini göstermek istediğini söyledi.

3. Kuyruk

Kuyruk (queue) gerçek dünyadaki bir sıraya benzer. Bilet almak için sıraya girdiğinizi ya da havaalanında check-in sırasında beklediğinizi düşünün. İnsanlar sırayla gelir: birinci, ikinci, üçüncü. İlk gelen önce işini bitirir ve sıradan ayrılır. En son gelenin sırası en sonda gelir ve o da en son çıkar.

Kuyruk bu yüzden "ilk gelen ilk çıkar, son gelen son çıkar" ilkesine göre çalışır. Hoca bu ilkeyi iki adla verdi:

- **FIFO** (first in, first out): ilk giren ilk çıkar.
- **LILO** (last in, last out): son giren son çıkar.

Kuyruk yapısı, bildiğimiz diziler gibi de bağlı listeler gibi de oluşturulabilir. Kuyrukta iki işlem vardır: **ekleme** (kuyruğa yeni bir elemanın gelmesi) ve **çıkarma** (işi bitenin kuyruktan ayrılması). Slayttaki şekilde de elemanlar sırayla girer ve ilk giren çıkar.

4. Yığın

Yığın (stack) kavramı programlama dillerinde, özellikle Alt Seviye Programlama dersinde çok karşınıza çıkacak. Register'larla ilgili bir işlem yapmak ya da bellekten bir bilgi alıp çıkarmak, hep yığın üzerinden yapılan işlemlerdir.

Yığını bir tarafı kapalı, bir tarafı açık bir kavanoz gibi düşünün. Önce birinci eleman gelir ve dibe yerleşir, sonra ikinci, üçüncü ve dördüncü üstüne yerleşir. İlk gelen ilk çıkabilir mi? Hayır; önünde o kadar eleman var ki üstündekiler boşalmadan çıkması imkânsızdır. Bu yüzden yığın kuyruktan farklı çalışır:

- **FILO** (first in, last out): ilk giren son çıkar.
- **LIFO** (last in, first out): son giren ilk çıkar.

4.1 Push, Pop ve Yığın Taşması

Yığında iki işlem vardır:

- **Push** (itmek): yığına veri atmak.
- **Pop**: yığından veri çekmek.

Hoca bunu kendi iş listesiyle anlattı: Yapılacaklar listesine sürekli iş geliyor, push, push, push; bir pop edilse hemen yeni bir push geliyor. İşler bitmeden yenileri gelmeye devam ederse yığın taşar. Buna **yığın taşması** (stack overflow) denir: yığında yer kalmamıştır, veriler tutulamaz ve kaybolur.

Yığın bir diziyi ters çevirmek için rahatça kullanılabilir. Elimizde $1, 2, 3, 4, 5$ dizisi olsun. Elemanlar sırayla push edilir: 1, 2, 3, 4, 5. Sonra sırayla pop edilir: 5, 4, 3, 2, 1. Dizi ters çevrilmiş olur.

Yığın da arka planda dizilerle ya da bağlı listelerle gerçeklenir. Önemli olan yığının boyutudur; yığın, onun için açılan dizinin boyutuyla sınırlıdır. Yığının kullanıldığı yerlere örnek olarak hoca arama algoritmalarını ve bu derste görülecek matematiksel ifadelerin bir biçimden başka bir biçime dönüştürülmesini verdi.

4.2 Yığın Göstergesi

Yığının nerede olduğunu gösteren bir işaretçiye ihtiyaç vardır: **yığın göstergesi** (stack pointer, SP). Yığının gözleri ya 1 bayt ya da 2 bayt uzunluğundadır.

- Her push işleminde veri yazılır ve SP bir üstteki boş gözü gösterir. Gözler 1 bayt ise SP 1 artar, 2 bayt ise (örneğin integer) SP 2 artar.
- Her pop işleminde en üstteki veri çekilir ve SP aşağı iner; 1 ya da 2 eksilir.

Kısacası SP, yığındaki iniş çıkışları gösteren işaretçidir.

> **Ders sonrası not:** Hoca her push'ta SP'nin arttığını anlattı. Bu, yığının yukarı doğru büyüdüğü bir model için doğrudur. x86 gibi birçok mimaride yığın aşağı doğru büyür: push'ta SP azalır, pop'ta artar. Alt Seviye Programlama dersinde bu ayrıntı önemlidir.

5. Infix, Prefix ve Postfix Gösterimleri

Yığının bir kullanım alanı, matematiksel ifadelerin dönüştürülmesidir. Günlük hayatta $(A+B)*C$ gibi bir ifade yazdığımızda anlamı "A ile B'yi topla, C ile çarp"tır. İşlem önceliği şöyledir:

- Önce parantez ve üs alma,
- sonra çarpma ve bölme,
- en son toplama ve çıkarma.

Günlük hayatta kullandığımız bu gösterim **ara ek** (infix) gösterimidir. "Fix" ek demektir, "in" içinde, ortada demektir. Burada A ve B **operand**, $+$ ve $*$ ise **operatördür**. Operatör iki operandın ortasında yer alıyorsa gösterim infix'tir.

İki gösterim daha vardır:

- **Ön ek** (prefix): "Pre" ön demektir. Operatör, operandların önüne yazılır. $A+B$'nin prefix'i $+\,A\,B$'dir: "A ile B'yi, önündeki operatör neyse o işleme tabi tut."
- **Son ek** (postfix): "Post" son demektir. Operatör operandların arkasına yazılır. $A+B$'nin postfix'i $A\,B\,+$'dır. $A-B$'nin postfix'i $A\,B\,-$'dir: "A'dan B'yi çıkar."

Prefix gösterime **Polonya gösterimi** (Polish notation), postfix gösterime **Ters Polonya gösterimi** (Reverse Polish notation) da denir. Hoca bu adların çok önemli olmadığını söyledi.

Dönüşümler kâğıt kalemle yapılabilir; ama kodlamak gerektiğinde yığınlar ya da ikili ağaçlar (binary tree) kullanılır. Infix'ten prefix'e, infix'ten postfix'e, prefix'ten postfix'e, postfix'ten prefix'e, prefix'ten ve postfix'ten infix'e her türlü dönüşüm, belirli kurallar çerçevesinde yığın kullanılarak yapılabilir.

6. İfadeyi İkili Ağaca Yerleştirme ve Dönüşüm Örnekleri

Hoca, öğrencilerden yanlarında kâğıt kalem bulundurmalarını ve örnekleri birlikte yapmalarını istedi.

6.1 Örnek 1: $(A+B)*(C/D)$

Önce ifade bir ikili ağaca yerleştirilir. En son yapılacak işlem çarpmadır, bu yüzden kökte $*$ durur. Sol alt ağaçta $+$ ve çocukları A, B; sağ alt ağaçta $/$ ve çocukları C, D vardır. Anlamı: A ile B'yi topla, C'yi D'ye böl, sonra bu iki sonucu çarp.

Prefix'e çevirme: Önce parantez içleri çevrilir. A ile B'nin toplamı $+\,A\,B$, C'nin D'ye bölümü $/\,C\,D$ olur. Sonra bu iki sonuç çarpılır; prefix olduğu için çarpı en öne gelir:

$$*\ +\,A\,B\,/\,C\,D$$

Postfix'e çevirme: A ile B'yi toplarız: $A\,B\,+$. C'yi D'ye böleriz: $C\,D\,/$. Bu ikisinin sonucunu çarparız; postfix olduğu için çarpı en sona gelir:

$$A\,B\,+\ C\,D\,/\,*$$

> **Ders sonrası not:** Hoca prefix'i yazarken bir an "postfix ise önüne çarpı gelecek" dedi. Prefix'te işaret öne, postfix'te sona gelir; burada yazılan prefix'tir.

Neden bu gösterimlere ihtiyaç duyulur? Bilgisayarın matematiksel işlemleri infix gösterimle yapması zordur. Derleyici (compiler) ifadeyi arka planda, bizden bağımsız olarak prefix ya da postfix gösterime çevirir ve işlemleri yığın üzerinde daha hızlı yapar. Bunun nedeni Bölüm 8'de görülecek.

6.2 Örnek 2: $A * B/C + (C/D - A)$

Hoca bu örneği önce öğrencilerin kendilerinin yapmasını istedi. İlk adım, matematikte bu ifadenin nasıl hesaplanacağını düşünmektir:

- Çarpma ve bölme aynı önceliktedir; soldan sağa gidilir. Önce A ile B çarpılır, sonra sonuç C'ye bölünür. B'yi C'ye bölüp sonra A ile çarpmak da aynı sonucu verir, ama soldan sağa uygulamak gerekir.
- Parantezin içinde önce C, D'ye bölünür, sonra bu sonuçtan A çıkarılır. D'yi A'dan çıkarıp sonra C'ye bölmek yanlıştır.
- En son iki sonuç toplanır.

Prefix: A ile B çarpılır: $* A\, B$. Bu sonuç C'ye bölünür: $/\ *\ A\, B\, C$. C, D'ye bölünür: $/\, C\, D$. Bu sonuçtan A çıkarılır: $-\, /\, C\, D\, A$. İki sonuç toplanır:

$$+\ /\ *\ A\, B\, C\ -/C\, D\, A$$

> **Hocanın vurgusu:** A'yı alıp çıkarmayı önce yapmayın. Hep soldan sağa gidiyorum. Bu çok önemli.

Postfix: A ile B çarpılır: $A\, B\, *$. Sonuç C'ye bölünür: $A\, B\ *\ C\ /$. C, D'ye bölünür: $C\, D\, /$. Bu sonuçtan A çıkarılır: $C\, D\, /\, A\, -$. İki sonuç toplanır:

$$A\, B\, *\, C\, /\ C\, D\, /A\ -\ +$$

Hoca burada bir uyarı yaptı: $A\, B\, *$'dan sonra C'yi yanlış yere koyarsanız anlam "C'yi $A * B$'ye böl" olur. Oysa istenen, $A * B$'yi C'ye bölmektir.

6.3 Örnek 3: $\dfrac{b^2 - 4ac}{2a}$

Bildiğimiz bir formüldür. Önce üs alma işaretine bakılır. Prefix ve postfix gösterimde üs alma yukarı ok ($\uparrow$) ile gösterilir.

- Prefix'te $\uparrow\ b\ 2$ yazarsanız anlamı b^2'dir.
- $\uparrow\ 2\ b$ yazarsanız anlamı 2^b'dir. Dikkat edin, operandların sırası önemlidir.
- b^2 yerine $*\, b\, b$ da yazılabilir. Ama ifade beşinci dereceden olsaydı beş tane b'yi yan yana çarpmak doğru olmazdı; üs işaretini kullanmak daha doğrudur.

Ağaç: En son yapılacak işlem bölmedir; kökte $/$ durur. Sol tarafta payda değil pay, yani $b^2 - 4ac$ vardır; sağ tarafta $2a$ vardır.

- Sağ alt ağaç: $*$ ve çocukları 2, a.
- Sol alt ağaç: $-$. Onun solunda $\uparrow$ ve çocukları b, 2; sağında $*$. Bu çarpının bir çocuğu 4, diğeri yine bir $*$ ve onun çocukları a, c'dir.

Hoca ağacı şöyle özetledi: a ile c'yi çarptım, sonra 4 ile çarptım. b'nin karesini aldım ve bundan çıkardım. 2'yi a ile çarptım ve böldüm. Böylece infix gösterim bir ikili ağaca geçirilmiş olur. Prefix ve postfix, ister ifadeye ister ağaca bakarak yazılabilir.

Prefix: b^2: $\uparrow\ b\, 2$. 4 ile a'yı çarp: $*\, 4\, a$; bunu c ile çarp: $*\ *\ 4\, a\, c$. İkisini birbirinden çıkar: $-\ \uparrow b\, 2\ *\ *\, 4\, a\, c$. 2 ile a'yı çarp: $*\, 2\, a$. Çıkarmanın sonucunu buna böl:

$$/\ -\ \uparrow b\, 2\ *\, *4\, a\, c\ *\ 2\, a$$

Postfix:

$$b\,2\uparrow\ 4\,a*c*\ -\,2\,a*\ /$$

Hoca aralara bıraktığı boşlukların zorunlu olmadığını, yalnız okumayı kolaylaştırdığını söyledi.

7. Sayısal Bir Örnek ve Üs Alma

Hoca bir de sayılarla örnek yaptı: $(3+2)^2+8/4*3$.

Prefix: 3 ile 2 toplanır, toplamın karesi alınır, 8 4'e bölünür, bölüm 3 ile çarpılır, sonra hepsi toplanır:

$$+\ \uparrow +3\,2\ 2\ *\,/8\,4\ 3$$

Postfix:

$$3\,2+2\uparrow\ 8\,4/3*\ +$$

Hoca "8 4" yazılırken bunun seksen dört mü yoksa sekiz ve dört mü olduğunu sordu: aralarında boşluk vardır; boşluk yazılmasa bile operatör ve operandların yerinden ne olduğu anlaşılır.

Değer (prefix'ten): $3+2=5$; $5^2=25$; $8/4=2$; $2\cdot 3=6$; $25+6=31$.

Değer (postfix'ten): $3+2=5$; $5^2=25$; $8/4=2$; $2\cdot 3=6$; $25+6=31$.

Üç gösterim de aynı sonucu vermek zorundadır; sonuç **31**'dir.

Hoca son olarak üs almada operand sırasını bir kez daha hatırlattı: $A\,B$ ile $B\,A$ farklıdır. Biri A'nın B'inci kuvveti, diğeri B'nin A'ıncı kuvvetidir.

8. Yığın ile İfade Değerlendirme

Slaytların bu kısmını, dersi birlikte veren Ahmet Hoca hazırlamıştı. Slaytlarda bu işlemlerin nasıl kodlanacağı ve yığında nasıl yürüdüğü sözde kodla (pseudocode) anlatılıyor.

> **Hocanın vurgusu:** Sizden kod istemeyeceğiz. Yalnız bunun yığın üzerinde nasıl çalıştığı hakkında bilginiz olsun.

8.1 Infix İfade: İki Yığın

İfade: $2*(5*(3+6))/15-2$.

Infix ifade değerlendirilirken iki yığın kullanılır: biri operandların, biri operatörlerin tutulduğu yığın. İfade soldan okunur:

- 2 gelir, operand yığınına atılır. $*$ gelir, operatör yığınına atılır. Açma parantezi gelir, o da operatör yığınına atılır.
- 5 operand yığınına, $*$ operatör yığınına atılır. Bir açma parantezi daha gelir, atılır.
- 3 operand yığınına, $+$ operatör yığınına, 6 operand yığınına atılır.

Buraya kadar bir sorun yoktur: operatörler bir yığına, operandlar bir yığına yerleşmiştir. İşlem kapama parantezi gelinceye kadar böyle devam eder.

- **İlk kapama parantezi** gelir: "Önceden açılmış bir parantez var, şimdi parantezin içindeki işlemleri yapayım." Operatör yığınının tepesinde $+$ vardır. $+$ iki operand ister: operand yığınından 6 ve 3 çekilir, $+$ da çekilir. $3+6=9$ hesaplanır ve 9 push edilir. Açma parantezi de yığından silinir.
- **İkinci kapama parantezi** gelir. Yine iki operand çekilir: 9 ve 5. Operatör yığınından $*$ çekilir. $5\cdot 9=45$ hesaplanır, 45 push edilir. Açma parantezi de kaldırılır. Operatör yığınında yalnız ilk $*$ kalır; operand yığınında 2 ve 45 vardır.
- $/$ gelir, operatör yığınına eklenir. 15 gelir, operand yığınına eklenir.

- $-$ gelir. Yeni bir operatör yığına konmadan önce, tepedeki operatörle öncelik karşılaştırılır. Tepede $/$ vardır ve önceliği daha yüksektir. Bu yüzden önce bölme yapılır: 15 ve 45 çekilir, $45/15 = 3$ hesaplanır ve 3 push edilir. $/$ yığından kaldırılır.
- Tepede şimdi $*$ vardır. $*$, $-$'den önceliklidir. 3 ve 2 çekilir, $2 \cdot 3 = 6$ hesaplanır, 6 push edilir.
- Artık $-$ operatör yığınına eklenir. 2 gelir, operand yığınına eklenir.
- İfade bittiğinde yığın boşaltılır: 2 ve 6 çekilir, $6 - 2 = 4$ hesaplanır. Sonuç **4**'tür.

Kural özetle şudur: Kapama parantezi gelince, açma parantezine kadar olan işlemler yapılır ve açma parantezi de yığından çekilir. Parantez dışında $+, -, *, /$ gibi bir işaret gelince öncelik kontrol edilir. Yığında daha öncelikli bir işlem varsa önce o yapılır, sonra düşük öncelikli işaret yığına konur.

> **Ders sonrası not:** Slayttaki algoritma $/$ geldiğinde yığındaki $*$'ı yapmadan $/$'yı üstüne koyuyor. Sola birleşimli işlemlerde yaygın kural, eşit öncelikteki işlemi önce yapmaktır: $(2 \cdot 45)/15$. Bu örnekte iki yol da 6 verir; ama tam sayı bölmesinde sonuç değişebilir.

8.2 Prefix İfade: Tek Yığın

İfade: $- \; / \; * \; 20 \; * \; 50 \; + \; 3 \; 6 \; 300 \; 2$.

Prefix ile çalışırken kural biraz farklıdır: önce ifade ters çevrilir (reverse). Ters çevrilmiş hâli soldan okunur: $2, 300, 6, 3, +, 50, *, 20, *, /, -$. Hoca, ters çevrilince "300"ün "003" ve "50"nin "05" gibi göründüğünü, ama algoritmanın sayıların ters yazıldığını bildiği için onları 300 ve 50 olarak aldığını söyledi.

- Sayı gelince push edilir: 2, 300, 6, 3.
- $+$ gelir: son iki değer, 3 ve 6 çekilir. $3 + 6 = 9$, 9 push edilir.
- 50 push edilir. $*$ gelir: 9 ve 50 çekilir, $50 \cdot 9 = 450$ push edilir.
- 20 push edilir. $*$ gelir: 20 ve 450 çekilir, $20 \cdot 450 = 9000$ push edilir.
- $/$ gelir: 9000 ve 300 çekilir, $9000/300 = 30$ push edilir.
- $-$ gelir: 30 ve 2 çekilir, $30 - 2 = 28$. Sonuç **28**'dir.

> **Ders sonrası not:** Hoca derste "20 ile 450'yi çarptı, 900" dedi ve "900'ü 300'e böldü, 30" diye devam etti. $20 \cdot 450 = 9000$'dir ve $9000/300 = 30$ olur. Hocanın söylediği sonuç (28) doğrudur; yalnız ara değer 9000 olmalıdır.

8.3 Postfix İfade: Tek Yığın

İfade: $2 \; 5 \; 3 \; 6 \; + \; * \; * \; 5 \; / \; 2 \; -$.

Postfix ifade soldan okunur. Burada da tek yığın vardır.

- 2, 5, 3, 6 sırayla push edilir.
- $+$ gelir: 6 ve 3 çekilir, $3 + 6 = 9$ push edilir.
- $*$ gelir: 9 ve 5 çekilir, $5 \cdot 9 = 45$ push edilir.
- Bir $*$ daha gelir: 45 ve 2 çekilir, $2 \cdot 45 = 90$ push edilir.
- 5 push edilir. $/$ gelir: 90 ve 5 çekilir, $90/5 = 18$ push edilir.
- 2 push edilir. $-$ gelir: 18 ve 2 çekilir, $18 - 2 = 16$. Sonuç **16**'dır.

> **Ek bilgi (derste anlatılmadı):** Tek yığınlı değerlendirmede operand sırası önemlidir. Postfix'te bir işaret gelince ilk çekilen değer sağ operand, ikinci çekilen sol operanddır (90 ve 5 çekilince işlem $90/5$'tir). Prefix ters çevrilip okunduğunda ise ilk çekilen değer sol operanddır (9000 ve 300 çekilince işlem $9000/300$'dür). Çıkarma, bölme ve üs almada bu sıra karıştırılırsa sonuç yanlış çıkar.

8.4 Neden Prefix ve Postfix Daha Hızlı?

Infix değerlendirmede iki yığın, prefix ve postfix değerlendirmede tek yığın kullanılır. Yığına yapılan her push ve her pop bir işlemdir ve süreyi uzatır. Derleyicilerin ifadeleri prefix ya da postfix biçime çevirerek işlemesinin nedeni budur.

9. Gösterimler Arası Dönüşümler

Slaytlarda infix'ten postfix'e ve infix'ten prefix'e dönüşümün yığınla adım adım nasıl yapıldığı da var. Hoca, kendi kâğıt kalemle anlattığı yöntemin slaytlarda yığın üzerinde gösterildiğini, bunları adım adım takip ederek her şeyin görülebileceğini söyledi. Bu kadar ayrıntıya gerek olmadığını, bunun nasıl gerçekleştiğini bilmenin yeterli olduğunu belirtti.

9.1 Postfix'ten Infix'e

Verilen postfix ifadede işaretler soldan sağa takip edilir. Her işaret, önündeki iki ifadeyi birleştirir:

- $B\,C\,/$: B, C'ye bölünür, B/C olur.
- Önündeki $-$: A'dan bu sonuç çıkarılır, $A - B/C$ olur.
- İlerlenir: $A\,K\,/$, yani A, K'ya bölünür: A/K.
- Sonra L ile çıkarma yapılır.
- En son iki sonuç çarpılır.

9.2 Prefix'ten Postfix'e

Aynı mantıkla alt ifadeler bulunur ve işaret sona taşınır. Hoca aynı ifadenin prefix hâlinden başladı:

- $/\,B\,C$ görülür: B, C'ye bölünecek; postfix'te $B\,C\,/$ yazılır.
- Önünde $-$ ve A vardır: bu bölüm A'dan çıkarılacak; $A\,B\,C\,/\,-$ yazılır.
- Çarpıya geçmeden diğer tarafa bakılır: A, K'ya bölünmüş: $A\,K\,/$. Bu sonuçla L arasında çıkarma vardır: $A\,K\,/\,L\,-$.
- En son iki taraf çarpılır: $A\,B\,C\,/\;-\;A\,K\,/\,L\;-\;*$.

Prefix'ten infix'e ve postfix'ten prefix'e dönüşüm de aynı mantıkla yapılır; slaytlarda bunların yığın üzerinde nasıl yapıldığı adım adım anlatılıyor. Hoca bunları kendi başınıza izlemenizi istedi.

10. Algoritma Analizine Giriş

Aradan sonra hoca algoritma analizine geçti. Bu konunun ayrıntıları da Algoritma Analizi dersinde görülecek; burada karmaşıklıktan ve yürütme zamanının (running time) nasıl hesaplandığından bahsedildi.

Bir algoritmada şunlar önemlidir:

- **Doğruluk** (correctness): Algoritmanın doğru çalışması.
- **Zaman verimliliği** (time efficiency).
- **Bellek kullanımı.**
- **Optimallik:** Algoritma en kısa zamanda en iyi çözümü vermelidir.

Analiz için iki yaklaşım vardır:

- **Teorik analiz:** Zaman, algoritmanın girdi boyutuna bağlı olarak ölçülür. Hoca "fonksiyon" derken aslında algoritmayı kastettiğini söyledi. Algoritmadaki karşılaştırmalar, döngüler ve atamalar fonksiyonun elemanları gibi düşünülür. Bunların tekrar sayılarından polinom biçiminde bir fonksiyon çıkarılır; bu, algoritmanın yürütme zamanıdır. Karmaşıklık da bu yürütme zamanı üzerinden hesaplanır.
- **Deneysel (kaba) analiz:** Bir giriş verisi seçilir, zaman tutulur, algoritma çalıştırılır, sonuç alınınca zaman durdurulur. "Kabaca bu kadar milisaniyede çalıştı" denir.

10.1 Temel İşlem ve Giriş Boyutu

Temel işlemler (basic operation) atamalar, döngüler ve karşılaştırmalardır. Asıl soru, algoritmanın çalışma süresini en çok hangi işlemin etkilediğidir. Örneğin sıralamada çalışma süresini en çok etkileyen işlem karşılaştırmadır: bir eleman diğerinden büyük mü, küçük mü?

Problem	Giriş boyutu	Temel işlem
n elemanlı dizide arama	Dizinin eleman sayısı n	Karşılaştırma
İki matrisi çarpma	Matrisin boyutu	Çarpma
Bir tam sayının asal olup olmadığını bulma	Sayının ikili (binary) gösterimdeki basamak sayısı	Bölme (mod alma)
Graf problemi	Düğüm ve kenar sayısı	Gezilen düğümler ve kenarlar

Matris çarpımında hoca, satır ve sütunlar çarpıldıktan sonra bir toplama da yapıldığını, ama temel işlemin çarpma olduğunu söyledi.

11. Algoritma Analizinin Temel Kavramları

Bu dersin kapsamında en çok üzerinde durulacak kavram yürütme zamanıdır ve $T(n)$ hesapları yapılacaktır.

- **Yürütme zamanı** (running time), $T(n)$: Algoritmanın işlevini yerine getirebilmesi için, temel kabul edilen işlemlerden kaç tane yürütülmesi gerektiğini gösteren matematiksel ifadedir. Kaç atama, kaç karşılaştırma yapıldığı n'e bağlı olarak çıkarılır. Bu $T(n)$, algoritmayı anlatan fonksiyon olur; karmaşıklık da bu fonksiyon üzerinden çıkarılır.
- **Alan maliyeti** (space cost): Algoritmanın işlevini yerine getirebilmesi için gereken bellek ihtiyacıdır. Örneğin yığın taşıyorsa yığını daha büyük açmak gerekir. Bu tür işlemler yapılmıyorsa alan maliyeti genelde pek hesaplanmaz.
- **Asimptotik ifade:** Bir problemin büyümesinin üst, alt ve ortalama sınırlarını belirler. Hoca bunu şöyle eşledi: en uzun zaman Big-O, minimum sınır Big-Omega, ortalama sınır Big-Theta. Ortalama derken "Big-O ile Big-Omega'yı topla, ikiye böl" diye düşünülmemelidir.
- **Alan karmaşıklığı** (space complexity): Eleman sayısı n'in büyük değerleri için bellek gereksiniminin nasıl arttığını gösteren asimptotik ifadedir.
- **Zaman karmaşıklığı** (time complexity): Algoritma yürütülürken geçecek zamanın nasıl arttığını gösteren asimptotik ifadedir. Örneğin karmaşıklık 2^n ise zaman karmaşıklığı çok yüksektir.

Kısacası bu kavramlar şunu söyler: elimde şöyle bir problem ve şu input var; şu kadar zamana, şu kadar alana ihtiyacım var. Hepsi birbiriyle örtüşen kavramlardır.

> **Hocanın vurgusu:** Biz genelde hep Big-O üzerinden çalışırız, çünkü olabilecek en kötü senaryoyu düşünmeliyiz. Daha iyi olursa hiçbir problem yok.

Hoca bunu doğrusal arama (linear search) ile örnekledi: Aranan eleman dizinin ilk elemanında bulunursa bu en iyi durumdur (best case) ve insanı çok yanıltır. Dizinin en sonunda bulunursa bu en kötü durumdur (worst case). Her zaman en kötü durum düşünülmelidir ki önümüzü görebilelim.

> **Ders sonrası not:** Hoca Big-O'yu "en kötü durum", Big-Omega'yı "en iyi durum", Big-Theta'yı "ortalama durum" diye eşledi. Doğrusu: Bu üçü fonksiyonlar için **üst sınır, alt sınır ve sıkı sınır** gösterimleridir. En kötü, en iyi ve ortalama durum ise ayrı kavramlardır; her durumun kendi O, Ω ve Θ'sı olabilir. Örneğin araya ekleme sıralamasının en kötü durumu $\Theta(n^2)$, en iyi durumu $\Theta(n)$'dir.

11.1 Yürütme Zamanı Nasıl Görünür?

Bir kod üzerinden yürütme zamanı hesaplandığında karşımıza n'e bağlı bir ifade çıkar; burada n bağımsız değişkendir ve işlem sayısı ona bağlıdır.

- $T(n) = 1$: sabittir. "Her şey toz pembe."
- $T(n) = \log n$: logaritmiktir; oldukça iyi bir sonuçtur.
- $T(n) = n$: doğrusaldır.
- $T(n) = n^2$: kareseldir.

Bunlar sırayla yürütme zamanının ya da karmaşıklığın arttığını gösterir. Hoca $n = 4$ ile gösterdi: $\log_2 4 = 2$, $n = 4$, $n^2 = 16$. Sayıların nasıl hızla arttığı görülür.

Hoca, işin ehli olunca yürütme zamanının artık hesaplanmadığını, algoritmaya bakıp yalnız döngüleri hesaba katarak karmaşıklığın gözle söylenebildiğini belirtti. "Ama şimdi yolun başındayız" diyerek örneklere geçti.

12. Yürütme Zamanı Hesabı: Ortalama ve En Küçük Eleman

Örneklerde algoritmanın ne yaptığının bir önemi yoktur; yalnız bir kod olarak bakılır.

12.1 Örnek 1: Ortalama Bulma

```
float BulOrta(float A[], int n) {
    float ortalama, toplam = 0;   /* 1 işlem */
    int k;
    for (k = 0; k < n; k++)       /* 1 + (n+1) + n = 2n+2 */
        toplam = toplam + A[k];   /* 2 işlem, n kez: 2n */
    ortalama = toplam / n;        /* 2 işlem */
    return ortalama;              /* 1 işlem */
}
```

Satır satır sayım:

- Değişken tanımlarında bir işlem yoktur. `toplam = 0` bir atamadır: 1 işlem.
- `for` döngüsü:
  - `k = 0` ataması bir kez yapılır: 1.
  - `k < n` karşılaştırması: k sıfırdan başlar; $k = 0, 1, \ldots, n-1$ için doğru, $k = n$ için yanlış çıkar. Toplam $n + 1$ kez yapılır.
  - `k++` artırması: yalnız k, n'den küçükken yapılır; n kez.
  - Döngü satırı toplamı: $1 + (n + 1) + n = 2n + 2$.
- Döngü gövdesinde bir toplama ve bir atama vardır: 2 işlem. Döngü içinde olduğu için n kez tekrar edilir: $2n$.
- `ortalama = toplam / n`: bir bölme ve bir atama, 2 işlem.
- `return`: 1 işlem. Hoca return'ü işlem sayıp saymamanın önemsiz olduğunu söyledi.

$$T(n) = 1 + (2n + 2) + 2n + 2 + 1 = 4n + 6$$

Hoca karşılaştırmanın $n + 1$ kez yapılmasını şöyle yorumladı: n'in 10 bin olduğu bir yerde bir eklesek ne olur, eklemesek ne olur? Ama $T(n)$ istendiğinde bu ayrıntılara dikkat edilir.

12.2 Örnek 2: En Küçük Elemanı Bulma

```
float BulEnkucuk(float A[], int n) {
    float enkucuk;
    int k;
    enkucuk = A[0];               /* 1 */
    for (k = 1; k < n; k++)       /* 1 + n + (n-1) = 2n */
        if (A[k] < enkucuk)       /* n-1 */
            enkucuk = A[k];       /* en kötü durumda n-1 */
    return enkucuk;               /* 1 */
}
```

- `enkucuk = A[0]`: 1 atama.
- `for` döngüsü: `k = 1` ataması 1 kez; `k < n` karşılaştırması n kez ($k = 1, \ldots, n$); `k++` artırması $n - 1$ kez, çünkü k'nın n'e eşit olduğu an alınmaz. Toplam $1 + n + (n - 1) = 2n$.
- `if` karşılaştırması döngü $n - 1$ kez döndüğü için en fazla $n - 1$ kez yapılır.
- `if` içindeki atama: Koşul doğru olduğu sürece yapılır. Kaç kez doğru olacağı bilinmediği için en kötü durum düşünülür: $n - 1$ kez.
- `return`: 1.

$$T(n) = 1 + 2n + (n-1) + (n-1) + 1 = 4n$$

Hocanın vurgusu: Yürütme zamanı, o algoritmanın içinde yapılan her bir işlemin en kötü şartlarda en fazla kaç kez tekrar edildiğinin hesabıdır. En iyi durumu bilemem; en kötü durumla gitmeliyim.

Ders sonrası not: Slayttaki fonksiyon `BulEnkucuk(float A[])` biçimindedir ve n parametre olarak verilmemiştir. Kodun derlenebilmesi için burada n parametre olarak eklendi. Bu iki fonksiyon `gcc -Wall` ile uyarısız derlendi.

13. Yürütme Zamanı Hesabı: Seçmeli Sıralama

Hoca "burada zor bir şey var" diyerek seçmeli sıralamanın (selection sort) yürütme zamanına geçti. Slaytta satırlar I'den VII'ye kadar numaralanmış, her satırdaki işlemler bir tabloda yazılmıştır.

```
public static int[] selectionsort(int[] A, int n) {
    int tmp; int min;
    for (int i = 0; i < n-1; i++) {        // I
        min = i;                           // II
        for (int j = i; j < n; j++) {      // III
            if (A[j] < A[min]) {           // IV
                min = j;                   // V
            }
        }
        tmp = A[i]; A[i] = A[min]; A[min] = tmp;   // VI
    }
    return A;                              // VII
}
```

- **I:** Bir atama (`i = 0`), bir karşılaştırma, bir artırma. Atama 1 kez, karşılaştırma n kez, artırma n kez alındı. Toplam $2n + 1$.
- **II:** Bir atama; dış döngünün içinde olduğu için n kez. Toplam n.
- **III:** Bir atama, bir karşılaştırma, bir artırma. `j = i` ataması, üstteki döngü kaç kez dönüyorsa o kadar yapılır: n kez. Karşılaştırma ve artırma ise n kez denemez, çünkü j, i'ye bağlıdır.

13.1 İç Döngünün Tekrar Sayısı

Hoca bunu $n = 5$ ile gösterdi. $i = 0$ iken j 0'dan başlayıp 5'e kadar gider; $i = 1$ iken 1'den, $i = 2$ iken 2'den, $i = 3$ iken 3'ten başlar. Her turda iç döngüde yapılan iş bir azalır: 5, 4, 3, 2, 1. Toplam:

$$5 + 4 + 3 + 2 + 1 = 15 = \frac{5 \cdot 6}{2}$$

Genel olarak 1'den n'e kadar sayıların toplamı $\frac{n(n+1)}{2}$'dir. Bu sayıları tek tek toplamak yerine n cinsinden yazmak çok daha önemlidir. Bu yüzden III satırındaki karşılaştırma ve artırma $\frac{n(n+1)}{2}$'şer kez yapılır:

$$\text{III} = n + 2 \cdot \frac{n(n+1)}{2} = n + n(n+1)$$

- **IV:** `if` karşılaştırması iç döngü kadar döner: $\frac{n(n+1)}{2}$.
- **V:** Atama. `if` içine girse de girmese de girmiş gibi düşünülür (en kötü durum): $\frac{n(n+1)}{2}$.
- **VI:** Üç atama (yer değiştirme).

- **VII:** `return`, 1.

Hoca anlatırken bir ara iç döngü toplamının ayrıca n ile çarpılacağını söyledi, sonra bunu düzeltti: $\frac{n(n+1)}{2}$ zaten bütün turların toplamıdır, n ile çarpılmaz.

Satır	İşlem	Tekrar	Toplam
I	1, 1, 1	$1, n, n$	$2n+1$
II	1	n	n
III	1, 1, 1	n, $\frac{n(n+1)}{2}$, $\frac{n(n+1)}{2}$	$n + n(n+1)$
IV	1	$\frac{n(n+1)}{2}$	$\frac{n(n+1)}{2}$
V	1	$\frac{n(n+1)}{2}$	$\frac{n(n+1)}{2}$
VI	1, 1, 1	n	$3n$
VII	1	1	1

13.2 Derste Gelen İki Soru

Soru: i sıfırdan başlayıp $n-1$'e kadar gidiyor. Dış döngünün n değil $n-1$ kez dönmesi gerekmiyor mu?

Cevap: Hoca önce şöyle cevap verdi: n'in 10 000 olduğunu düşünün; 10 000 kez mi, 9 999 kez mi kullanıldığı fark etmez. Karmaşıklığa geçince sabit olarak eklenen sayılar ihmal edilecek, hatta düşük dereceli n'li terimler bile ihmal edilecek; en yüksek dereceli terim kullanılacak. n^2 ile n^3 küçük sayılarda yakın olabilir, ama $n = 10\,000$ için aralarında "dağlar kadar fark" vardır; böyle bir durumda n^2 bile ihmal edilir. Sonra $n = 5$ ile birlikte kontrol ettiler: koşul $i < 4$ olduğu için $i = 0, 1, 2, 3$ değerleri alınır; yani soru doğrudur, $n-1$ kez döner. Hoca $n-1$ ile uğraşmayıp doğrudan n aldığını, asıl önemli kısmın n^2'li terimleri getiren iç döngü olduğunu söyledi.

Soru: VI satırındaki işlemler birinci `for`'un içinde olduğu için n kez tekrarlanmıyor mu?

Cevap: Hoca kontrol etti ve öğrencinin haklı olduğunu söyledi. Slaytta VI satırının toplamı 3 yazılmıştı; doğrusu $3n$'dir. Bu düzeltmeyle $6n$'lik terim $9n$, sabit terim de 5 yerine 2 olur. Hoca, n^2'nin yanında bunun önemi olmasa da slaytın düzeltilmesini istedi ve öğrenciye teşekkür etti.

Slayttaki ilk sonuç $T(n) = 2n^2 + 6n + 5$ idi. Düzeltilmiş toplam:

$$T(n) = (2n+1) + n + \big(n + n(n+1)\big) + n(n+1) + 3n + 1 = 2n^2 + 9n + 2$$

Karmaşıklık $O(n^2)$'dir.

Hoca bu soru vesilesiyle slaytların yeni hazırlandığını, hata görülürse söylenmesini istedi.

> **Ders sonrası not:** Dış döngü $i = 0, \dots, n-2$ olmak üzere $n-1$ kez döner. İç döngünün tam toplamı $n + (n-1) + \cdots + 2 = \frac{n(n+1)}{2} - 1$'dir. Slayttaki $\frac{n(n+1)}{2}$ bir üst yaklaşımdır. Karmaşıklık değişmez: $O(n^2)$. Ayrıca iç döngünün $j = i + 1$'den başlaması yeterlidir; $A[i]$'yi kendisiyle karşılaştırmaya gerek yoktur.

14. Yürütme Zamanı Hesabı: Matris Toplama

$n \times m$ boyutlu iki matrisin toplamı, iç içe iki döngüyle yapılır.

- **Dış döngü:** Atama 1 kez, karşılaştırma $n+1$ kez, artırma n kez: $2n+2$.
- **İç döngü:** Atama 1, karşılaştırma $m+1$, artırma m kez; yani bir turda $2m+2$. Dış döngü n kez döndüğü için $(2m+2) \cdot n = 2mn + 2n$.
- **Gövde:** Bir toplama ve bir atama, 2 işlem. İç döngüden dolayı m ile, dış döngüden dolayı n ile çarpılır: $2mn$.

$$T(n, m) = (2n+2) + (2mn + 2n) + 2mn = 4mn + 4n + 2$$

Sonuç tek bir değişkenle ifade edilmelidir. $m = n$ alınırsa:

$$T(n) = 4n^2 + 4n + 2$$

Hoca, karşılaştırmayı $n + 1$ yerine n diye de yazabileceğini, çünkü bu sabitlerin ihmal edileceğini söyledi. $T(n)$ istendiğinde bunlara dikkat edilir; ama "karmaşıklığını bul" dendiğinde bunların hiçbir önemi kalmaz.

Bu fonksiyonun Big-O'su n^2'dir. Big-O, fonksiyondaki en yüksek mertebeye sahip terimdir; önündeki 4 gibi sabitler bile alınmaz.

> **Hocanın vurgusu:** Bu işin raconunu bilince işlemlere tek tek bakmıyorum. İç içe iki döngü varsa karmaşıklık n^2'dir. Bir döngü daha eklersem, uzunluğu ne olursa olsun, en kötü şartlarda hepsini eşit kabul ederim; üç döngü varsa karmaşıklık n^3'tür.

15. Fonksiyonların Büyümesi

Hem bilgisayar biliminde hem matematikte bir fonksiyonun ne kadar hızlı büyüdüğü sıklıkla incelenir. Elimizde bir $T(n)$ fonksiyonu vardır. x ekseninde girdi, y ekseninde zaman olacak şekilde çizilir ve gelişimine bakılır. Aynı problemi çözen iki farklı algoritma varsa hangisinin daha verimli olduğuna buna göre karar verilir. Fonksiyonun büyümesi (growth of functions) asimptotik gösterimlerle incelenir.

Karmaşıklık: Bir algoritmanın üzerinde işlem yapacağı veri kümesinin eleman sayısı arttığında, yürütme zamanı maliyetinin nasıl değişeceğini gösteren asimptotik ifadedir.

Yürütme zamanı $T(n)$ girdinin boyutu n ile ilişkilidir. Big-O, Big-Theta ve Big-Omega, n ile $T(n)$ arasındaki ilişkiyi verir. Yürütme maliyeti hesaplanarak zaman karmaşıklığı, bellek maliyeti hesaplanarak alan karmaşıklığı bulunur. Algoritmalarda genelde $T(n)$ hesaplanarak zaman karmaşıklığı bulunur.

Sık karşılaşılan fonksiyonlar, küçükten büyüğe:

$$1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$$

$\log n$ en düşük karmaşıklıktır. Sonra n gelir; $\log n$ ile n çarpılınca biraz daha artar; n^2, n^3 ve 2^n daha büyüktür; hocanın deyişiyle $n!$ "işin laçkasının çıkarılmış hâli"dir.

Slayttaki tabloda n 10'dan 10^6'ya çıkarken her fonksiyonun aldığı değerler var. $\log n$'deki değişim çok küçüktür. n'deki değişim doğrusaldır. $n \log n$'de büyüme başlar, n^2'de gittikçe korkunçlaşır; sonrakiler artık hesaplanamaz bile. Böyle karmaşıklıklarla bir problem çözülemez; bunlar ileride polinom zamanda çözülemeyen problemler olarak karşımıza çıkacak. Hoca grafiği şöyle okudu: sabit fonksiyon hiç büyümez ve en ideal olanıdır; n biraz artar; $n \log n$ ve n^2 daha çok artar; $n!$ ise n çok az büyüdüğünde birden pik yapar. Fonksiyonun büyümesi, input sayısına göre yapılan işlem sayısının ne kadar arttığını gösterir.

16. En Kötü, En İyi ve Ortalama Durum

Algoritmalar en kötü durum, en iyi durum ve ortalama durum için analiz edilebilir. Ama genelde her zaman en kötü duruma göre analiz edilmelidir; olabilecek en kötü şartları düşünüp ona göre hazırlanmak gerekir. Çünkü algoritma aynı kalsa da inputlar değişir: bazı girişlerde çok hızlı, bazılarında ağır çalışır.

- **En kötü durum** (worst case): Algoritmanın en fazla sürede çalıştığı analiz türüdür. Çalışma zamanı için bir üst sınırdır ve algoritmanın bundan daha uzun sürmeyeceğini garanti eder. Bazı algoritmalarda en kötü durum sık görülür: aramada aranan öğe genellikle dizide yoktur ve döngü n kez çalışır. Aranan elemanın dizide olmaması, işlemin boşuna yapılması demektir.
- **En iyi durum** (best case): Bir alt sınırdır. Algoritmanın en kısa sürede ve en az adımda çalıştığı giriş durumudur. Aramada aranan terimin ilk sırada bulunması gibi. Hoca, "Kaç kere insanın karşısına böyle bir şans gelebilir?" diyerek bunun pek dikkate alınmadığını söyledi.

- **Ortalama durum** (average case): Algoritmanın ortalama sürede ve ortalama adımda çalıştığı giriş durumudur. Bu ortalama, en iyi ve en kötü durumun ortalaması değildir; ayrı hesaplanır.

17. Algoritma Seçimi: Büyüme Hızına Göre Karar

Giriş boyutu n olan bir problem için A ve B adlı iki algoritma olsun. Yürütme zamanları $T_A(n)$ ve $T_B(n)$ hesaplanmış ve grafiğe çizilmiş olsun.

İlk grafikte T_B, input ne kadar artarsa artsın belli bir değeri hiç geçmiyor. Bu durumda B seçilir. Yürütme zamanı en düşük olan algoritma seçilir; bu aynı zamanda karmaşıklığı en düşük olandır.

İkinci durumda $T_A(n) = 50n$ ve $T_B(n) = n^2$'dir. Grafiğe bakınca n 50'ye gelene kadar B daha iyi çalışıyor gibi görünür. Ama input 50'yi geçince B'nin çalışma süresi birden artar. A'nın çalışma zamanı ise artan n değerlerinde daha yavaş ilerler. Bu durumda A seçilir.

Hoca bir istisna da söyledi: Girişin bu sınırın ötesine geçmeyeceği garanti ise B seçilebilir. Ama böyle bir garanti yoksa ve "her şey olur" denirse A seçilmelidir.

Hoca şunu da ekledi: Örneklerde ya da sınavda 10 ya da 20 elemanlı diziler kullanılır, ama gerçek dünya böyle değildir. Eleman sayısının küçük olduğu durumlar mümkündür, fakat birçok uygulama için geçerli değildir.

18. Asimptotik Gösterim ve Karmaşıklık Sınıfları

Asimptotik gösterim, eleman sayısı n sonsuza giderken bir algoritmayı benzer işi yapan algoritmalarla karşılaştırmak için kullanılır. Çalışma zamanları $T_1(n)$ ve $T_2(n)$ olan iki algoritmadan hangisinin daha iyi olduğunu belirlemek için:

- **Big-O:** asimptotik üst sınır,
- **Big-Omega:** asimptotik alt sınır,
- **Big-Theta:** asimptotik alt ve üst sınır kullanılır.

Hangi algoritma kitabına bakılırsa bakılsın, genelde algoritmanın Big-O karmaşıklığı verilir. Hoca, $T(n)$ ile bulunan polinomu bu kısımda x cinsinden yazıp $f(x)$ diye adlandıracaklarını söyledi. $O(f(x))$ asimptotik üst sınırı, $\Omega(f(x))$ alt sınırı, $\Theta(f(x))$ ise ikisini birden verir.

Slaytta karmaşıklık sınıfları "mükemmel"den "korkunç"a doğru renklerle sıralanmış bir grafikte gösteriliyor; $n!$ için "don't ask" (hiç sorma) yazıyor. Bu grafikte:

- İkili arama (binary search) $\log n$ bölgesindedir; çok iyidir. Sürekli yarıya bölünerek gidildiği için $\log n$'dir. Doğrusal aramanın (linear search) karmaşıklığı ise $O(n)$'dir. İkili aramanın doğrusal aramadan daha iyi olmasının nedeni budur.
- Quicksort $n \log n$ bölgesindedir; diğer sıralamalardan daha iyi olmasının nedeni karmaşıklığının daha az olmasıdır.
- Seçmeli sıralama bile n^2 karmaşıklığa sahiptir.
- 2^n ve $n!$ ütopik karmaşıklıklardır; bu algoritmaları bilgisayarlarda çalıştırmak zordur. Böyle bir problemi doğrudan çözmek yerine parçalara bölerek çözmek ya da belirli bir veri için çözmek daha mantıklıdır.

Karmaşıklık sınıflarının adları: sabit (constant), logaritmik (logarithmic), doğrusal (linear), doğrusal-logaritmik (linearithmic, $n \log n$), polinom (polynomial), üstel (exponential) ve faktöriyel (factorial). Hoca bu terimlerin bilinmesini istedi.

Slaytta bir de problem boyutu n 10'dan 10^6'ya giderken, her karmaşıklık için kullanılan bit işlemlerinin süresini veren bir tablo var. Örneğin $\log n$ satırında sürelerin çok küçük kaldığı görülür.

19. Big-O Tanımı ve İspat Örneği

Hoca bu kısımda bazı ispatlar olduğunu, bunlar üzerinde çok durmak istemediğini, ayrıntıların Algoritma Analizi dersinde görüleceğini söyledi. Amaç, o derse gelindiğinde "evet, böyle şeylerden bahsetmiştik" diyebilmektir.

19.1 Tanım

f ve g, tam sayılar kümesinden ya da reel sayılar kümesinden reel sayılara tanımlı iki fonksiyon olsun. Öyle C ve k sabitleri varsa ki

$$|f(x)| \leq C\,|g(x)|, \qquad x > k$$

sağlanır, o zaman $f(x) = O(g(x))$ yazılır. Bu koşulu sağlayan C ve k'ye **tanık** (witness) denir. Yani bir tanık bulunur; öyle C ve k değerleri seçilir ki koşul sağlansın. Sağlanıyorsa f'nin karmaşıklığı en fazla g'nin karmaşıklığı kadardır. Hoca kitaplarda C'nin sıfırdan büyük olması gerektiğinin de yazıldığını hatırlattı.

19.2 Hocanın Benzetmesi

Geçen dönem 10 ders alıp 10 dersin vizesine hazırlanmış biri olsun. g fonksiyonu, 10 ders için harcanan hazırlık süresini verir. f fonksiyonu ise 7 ders alan birinin hazırlık süresidir. Bu iki kişinin aynı kapasitede olduğunu düşünelim. 10 ders alan kişinin ne kadar süre ayırdığını biliyorsam, 7 ders alan kişiye "bu sürenin üstüne çıkamazsın" diyebilirim. En kötü durumda ancak bu kadar sürede yapar ya da daha kısa sürede yapar. g önceden bilinen, bütün bilgileri elimizde olan bir problemdir; f ona benzeyen ama şartları biraz farklı bir problemdir.

19.3 Örnek: $f(x) = x^2 + 2x + 1$ için $O(x^2)$

Burada $g(x) = x^2$'dir. Bir fonksiyonun yürütme zamanı x^2 ise karmaşıklığı da x^2'dir. Gösterilmesi gereken: $x > k$ için $f(x) \leq C\,g(x)$.

$k = 1$ **seçimi:** $x > 1$ ise $x < x^2$ ve $1 < x^2$ olur. En kötü durumu düşünerek $f(x)$'teki x yerine x^2, 1 yerine de x^2 konur:

$$x^2 + 2x + 1 \leq x^2 + 2x^2 + x^2 = 4x^2$$

Tanıklar $C = 4$, $k = 1$'dir. Hoca $x = 2$ ile kontrol etti: $f(2) = 4 + 4 + 1 = 9$ ve $4 \cdot 2^2 = 16$; $9 \leq 16$ sağlanır. Bundan sonra hangi değer konursa konsun koşulun sağlandığı görülür.

$k = 2$ **seçimi:** $x > 2$ ise $2x < x^2$ ve $1 < x^2$ olur. $2x$ yerine x^2 konur:

$$x^2 + 2x + 1 \leq x^2 + x^2 + x^2 = 3x^2$$

Tanıklar $C = 3$, $k = 2$'dir. Hoca $x = 3$ ile kontrol etti: $f(3) = 9 + 6 + 1 = 16$ ve $3 \cdot 3^2 = 27$; $16 \leq 27$ sağlanır.

Slayttaki grafikte $g(x)$, $f(x)$ ve $C\,g(x)$ birlikte çiziliyor. Gösterilen şey, C çarpanıyla çarpılınca $C\,g(x)$'in $f(x)$'in üstünde kaldığıdır.

> **Ders sonrası not:** Hoca $k = 2$ ispatında "x 2'den büyükse $2x$ de x^2'den büyüktür" dedi. Doğrusu tersidir: $x > 2$ ise $x^2 > 2x$. İspatta da $2x$ yerine x^2 konarak bu doğru yön kullanıldı.

20. Sıralı Arama ve Bilinen Algoritmaların Karmaşıklığı

20.1 Sıralı Arama

Slayttaki algoritma bir sıralı aramadır (sequential search). En kötü, en iyi ve ortalama durumu sorulduğunda iki yol vardır: Ya $T(n)$ hesaplanır, ya da algoritmaya bakılır. Burada bir `while` döngüsü vardır. Hoca "`if`'leri hesaba katma, en önemlisi döngü" dedi: döngü n kez dönüyorsa en kötü durum n'dir.

- **En iyi durum:** Aranan eleman birinci sıradadır; 1 karşılaştırma.
- **En kötü durum:** Aranan eleman n'inci sıradadır; n karşılaştırma.

- **Ortalama durum:** Eleman ya birinci, ya ikinci, ya üçüncü, …, ya da n'inci sırada bulunur. Bunlar toplanıp eleman sayısı n'e bölünür:

$$\frac{1+2+\cdots+n}{n} = \frac{n(n+1)/2}{n} = \frac{n+1}{2}$$

Hoca burada sonucun en iyi ve en kötü durumun ortalamasına eşit çıktığını, ama bunun bir tesadüf olduğunu, ortalama durumun öyle hesaplanmadığını vurguladı.

$T(n)$ ile hesaplansaydı, n'e bağlı "bir şey çarpı n artı bir şey" biçiminde bir ifade çıkacaktı; en büyük katlı terim n olduğu için karmaşıklık n olacaktı. Kaba tabirle: bir `while` var, n kez dönüyor; karmaşıklık n'dir. `if`'lerin hepsi en kötü koşulda n kez girilmiş kabul edilir ve yalnız toplamaya eklenir. Hoca öğrencilerden bu algoritmanın $T(n)$'ini kendilerinin hesaplamasını istedi (bkz. Örnek Sorular).

20.2 Bilinen Karmaşıklıklar

Slaytta karmaşıklıklar açıklamalarıyla verilmiş:

- $O(1)$**:** Veri giriş boyutundan bağımsız gerçekleşen işlemler; n'e hiç bağlı olmayan bir işlem. Örneğin diziye bir eleman eklemek.
- $O(\log n)$**:** İkili arama. İkiye bölerek gidiliyorsa işin içinde mutlaka logaritma vardır.
- $O(n)$**:** Doğrusal bir dizide arama.
- $O(n \log n)$**:** Bir problemi küçük parçalara bölerek çözen böl ve fethet (divide and conquer) algoritmaları. Quicksort bunlardan biridir: diziyi ikiye böler, parçaları kendi arasında sıralarken onları da ikiye böler. Hep bölme olduğu için $\log n$, bu n kez yapıldığı için n ile çarpılır.
- $O(n^2)$**:** Sıralamaların büyük çoğunluğu: kabarcık (bubble), shell ve araya ekleme (insertion) sıralaması.
- $O(n^3)$**:** Hoca "belki üç matrisin birbiriyle çarpımı" ve "üç boyutlu düzlemlerde işlem yaparken" dedi.
- $O(2^n)$**:** Hanoi kuleleri. Üç çubuk ve farklı büyüklükte diskler vardır; diskler çubuklar kullanılarak bir çubuktan diğerine aynı düzende taşınacaktır. Bu problemin algoritmasının karmaşıklığı 2^n'dir.

  Ders sonrası not: Quicksort'un $O(n \log n)$ karmaşıklığı ortalama durumdur; en kötü durumu $O(n^2)$'dir. Shell sort'un karmaşıklığı seçilen aralık dizisine bağlıdır; örneğin bazı dizilerle $O(n^{3/2})$ veya $O(n \log^2 n)$ olur. Kübik örnek için: klasik yöntemle iki $n \times n$ matrisin çarpımı zaten $O(n^3)$'tür; üç matris gerekmez.

  Ek bilgi (derste anlatılmadı): Hanoi kulelerinde n diski taşımak için en az $2^n - 1$ hamle gerekir. Bu yüzden karmaşıklık $O(2^n)$'dir.

21. Big-Omega ve Big-Theta

21.1 Big-Omega

Big-Omega'da koşul tersine döner: $f(x) \leq C\, g(x)$ yerine

$$|f(x)| \geq C\, |g(x)|, \qquad x > k$$

olur. Yine $x > k$ olmak üzere C ve k sabitleri bulunmaya çalışılır. Bulunabiliyorsa $f(x) = \Omega(g(x))$'tir.

Slayttaki örnek: $f(x) = 8x^3 + 5x^2 + 7$ ve $g(x) = x^3$. $C = 8$ alınır; bütün pozitif x değerleri için

$$8x^3 + 5x^2 + 7 \geq 8x^3$$

sağlanır. Yani $f(x) = \Omega(x^3)$'tür.

21.2 Big-Theta

Big-Theta'da $f(x)$, $g(x)$'in hem altında hem üstünde sınırlanır. C_1 çarpanı g'yi f'den küçük, C_2 çarpanı büyük yapar:

$$C_1 \left|g(x)\right| \leq \left|f(x)\right| \leq C_2 \left|g(x)\right|, \qquad x > k$$

Bu koşulu sağlayan C_1, C_2 ve k bulunabiliyorsa $f(x) = \Theta(g(x))$'tir.

22. Limit ve Tümevarım ile Karmaşıklık

Ahmet Hoca slaytlara iki örnek daha eklemişti. Bu örnekler, karmaşıklığın yalnız tanıkla değil, farklı yöntemlerle de gösterilebileceğini anlatıyor.

22.1 Limit ile

$f(x) = 4x^3 + 3x^2$ ve $g(x) = x^4$ için $\dfrac{f(x)}{g(x)}$'in limiti alınır:

$$\lim_{x \to \infty} \frac{4x^3 + 3x^2}{x^4} = \lim_{x \to \infty} \left(\frac{4}{x} + \frac{3}{x^2} \right) = 0$$

Limit sıfır olduğu için $f(x) = O(g(x))$, yani $4x^3 + 3x^2 = O(x^4)$'tür. Hoca, $C = 7$ ve bir k değeri seçilerek tanımın da sağlandığının görülebileceğini söyledi: $x \geq 1$ için $4x^3 + 3x^2 \leq 4x^4 + 3x^4 = 7x^4$.

> **Ders sonrası not:** Hoca limitin "x sıfıra giderken" alındığını söyledi; limit $x \to \infty$ iken alınır. Ayrıca limit 0 çıkınca $f(x) = O(g(x))$ olur, ama bu sıkı bir sınır değildir. Hoca "f'nin karmaşıklığı x^4'tür" dedi; $4x^3 + 3x^2$'nin sıkı sınırı $\Theta(x^3)$'tür, x^4 yalnız bir üst sınırdır.

22.2 Tümevarım ile

İlk bölümlerde görülen tümevarım (induction) ile de karmaşıklık gösterilebilir. Soru: $n > 6$ olan tam sayılar için $3^n < n!$ olduğunu ispatlayınız. Burada $f(n) = 3^n$, $g(n) = n!$'dir.

- **Taban adımı:** $n = 7$ için $3^7 = 2187 < 7! = 5040$. Doğrudur.
- **Tümevarım hipotezi:** $n = k$ için $3^k < k!$ doğru kabul edilir.
- **Tümevarım adımı:** $n = k + 1$ için doğru mu diye bakılır. $(k+1)!$'den bir eleman dışarı alınıp $(k+1) \cdot k!$ yazılır:

$$3^{k+1} = 3 \cdot 3^k < 3 \cdot k! < (k+1) \cdot k! = (k+1)!$$

Son eşitsizlik $k \geq 7$ için $k + 1 > 3$ olduğundan doğrudur (ara adım eklendi).

Sonuç olarak $3^n = O(n!)$ denebilir.

> **Ders sonrası not:** Koşulun $n > 6$ olmasının nedeni, $n = 6$ için $3^6 = 729 > 6! = 720$ olmasıdır; eşitsizlik ilk kez $n = 7$'de sağlanır.

22.3 Karmaşıklık Tabloları

Slaytlarda başvuru için birkaç tablo da var:

- Arama algoritmaları ve bunların zaman ve alan karmaşıklıkları. Doğrusal arama, kaba kuvvet (brute force) olarak geçer. Kaba kuvvet algoritmalar, olabilecek bütün durumları test eden algoritmalardır. İkili arama da bilinir; diğerlerinin şimdilik bilinmesi gerekmez.
- Sıralama algoritmaları. Bu derste kabarcık ve araya ekleme sıralaması gösterildi; diğerleri ileride görülecek. Tablolarda zaman ve alan karmaşıklığı verilir; en çok kullanılan en kötü durumdur.

- Heap kullanan algoritmaların karmaşıklıkları.
- Graf işlemleri: düğüm ya da kenar ekleme, çıkarma, listeleme ve sorgulama (query) işlemlerinin karmaşıklıkları.

Hoca, bu tablolardaki bir algoritmanın kodunu internetten bulup kendi yaptığı gibi $T(n)$'ini çıkarmanızı ve tablodaki karmaşıklığı elde edip etmediğinize bakmanızı önerdi.

> **Ders sonrası not:** Hoca "heap de dinamik olarak oluşturulmuş bir stack'tir" dedi. Veri yapısı olarak heap, her düğümün çocuklarından büyük (veya küçük) olduğu tam ikili ağaçtır. Bellekteki "heap" bölgesi ise dinamik bellek ayırma alanıdır ve yığından (stack) ayrı bir bölgedir.

23. Birden Fazla Parçadan Oluşan Fonksiyonların Karmaşıklığı

Bir algoritma birden fazla alt programdan oluşabilir: Bir programın içinde birçok prosedür vardır. Bu durumda programın karmaşıklığına nasıl karar verilir?

23.1 Kural

f_1'in karmaşıklığı $O(n^c)$, f_2'nin karmaşıklığı $O(n^d)$ olsun ve $d > c > 1$ olsun. Bütün karmaşıklıklar yazılır ve **en büyüğü** alınır. Çünkü her zaman en kötü durum alınır; burada $d > c$ olduğundan karmaşıklık $O(n^d)$'dir.

- Toplanan parçalar için en büyük karmaşıklık alınır: $O(f) + O(g) = O(\max(f, g))$.
- Çarpılan parçalar için karmaşıklıklar çarpılır: $O(f) \cdot O(g) = O(f \cdot g)$.

Hoca iki örnekle gösterdi:

- Birinin karmaşıklığı $O(n)$, diğerininki $O(\log n)$ olsun. Sıralama $1, \log n, n, n \log n, n^2, n^3, 2^n, n!$ idi; en kötü olan n'dir. Sonuç $O(n)$.
- Biri $O(2^n)$, diğeri $O(n)$ ise sonuç $O(2^n)$'dir.
- İkisi aynıysa yalnız biri yazılır.

23.2 Örnek 1: $f(n) = 3n \log(n!) + (n^2 + 3) \log n$

Bu, karışık bir kodun $T(n)$'i gibi düşünülebilir. Önce sol taraf, sonra sağ taraf ayrı ayrı incelenir.

Sol taraf: $3n \log(n!)$**.** Arada çarpı olduğu için $3n$ ve $\log(n!)$ ayrı ayrı bulunur.

- $3n$: $T(n) = 3n$ ise sabit sayıların hiçbir önemi yoktur; en büyük üslü n terimi alınır. $n^2 + 2n + 1$'de n^2'nin yanında diğerlerinin önemi olmadığı gibi. Sonuç $O(n)$.
- $\log(n!)$: $n! \leq n^n$ olduğu için $\log(n!) \leq \log(n^n) = n \log n$. Logaritmada üs öne düşer. Sonuç $O(n \log n)$.
- Çarpım: $O(n) \cdot O(n \log n) = O(n^2 \log n)$.

Sağ taraf: $(n^2 + 3) \log n$**.** Çarpım dağıtılır: $n^2 \log n + 3 \log n$. Toplamda en büyük olan alınır: $O(n^2 \log n)$.

Toplam: İki tarafın da karmaşıklığı $n^2 \log n$'dir. Arada artı olduğu için en büyüğü alınır; ikisi aynı olduğundan:

$$f(n) = O(n^2 \log n)$$

> **Ders sonrası not:** Hoca "$\log n!$, n^n'dir" dedi. Doğrusu, $n! \leq n^n$ olduğu için $\log(n!) \leq n \log n$ eşitsizliğidir. Bu, $O(n \log n)$ sonucu için yeterlidir. Aslında $\log(n!) = \Theta(n \log n)$'dir.

23.3 Örnek 2: $f(x) = (x + 1) \log(x^2 + 1) + 3x^2$

- $x + 1$: Sabitlerin önemi yoktur; $O(x)$.

- $\log(x^2+1)$: Sabit atılınca $\log x^2$ kalır. Üs öne düşer: $2\log x$. Sabit 2'nin de önemi yoktur: $O(\log x)$.
- Arada çarpı olduğu için çarpılır: $O(x \log x)$.
- $3x^2$: Sabit atılır; $O(x^2)$.
- Arada artı olduğu için en büyüğü alınır. $x \log x$ ile x^2'den büyük olan x^2'dir.

$$f(x) = O(x^2)$$

n cinsinden yazılırsa $O(n^2)$'dir. Hoca slaytta iki yerde düzeltme gerektiğini (x teriminin eksik yazıldığı yer ve seçmeli sıralamadaki $3n$) söyledi ve slaytları indirenlerin yeni sürümü indirmesini istedi.

> **Ek bilgi (derste anlatılmadı):** $\log(x^2+1) = O(\log x)$ sonucunda "sabiti atmak" doğrudan değil, bir eşitsizlikle gerekçelendirilir: $x > 1$ için $x^2+1 \leq 2x^2$ olduğundan $\log(x^2+1) \leq \log 2 + 2\log x$ olur ve bu $O(\log x)$'tir.

24. Yinelemeli Algoritmaların Analizi

Hoca rekürans bağıntılarına gelmeden önce bu kısmı bitirmek istediğini söyledi.

Yinelemeli (iterative) fonksiyon, özyinelemeli olmayan fonksiyondur. **Özyinelemeli (recursive) fonksiyon** ise kendi kendini çağıran algoritmadır. Şimdiye kadar yapılan bütün algoritmalar yinelemeliydi.

Yinelemeli algoritmaların analizi şu sırayla yapılır: girdi boyutu belirlenir, temel işlemler belirlenir, en iyi, en kötü ve ortalama durum analizleri yapılır, n'e bağlı olarak yürütme zamanı hesaplanır ve buna bağlı olarak asimptotik gösterimlerle Big-O, Big-Theta ve Big-Omega çıkarılır.

24.1 Araya Ekleme Sıralaması

Araya ekleme sıralaması (insertion sort), "araya sokma" algoritmasıdır. Algoritma her zaman $j = 2$'den, yani ikinci elemandan başlar. O indisteki eleman bir anahtar (key) değişkeninde saklanır. Sonra $i = j-1$'den başa doğru bakılır: anahtardan büyük olan elemanlar birer sağa kaydırılır. Başa kadar gidilir ve anahtar açılan yere yerleştirilir.

Dizi: $11, 7, 15, 3, 16, 13$.

- **Anahtar 7:** 7, 11 ile karşılaştırılır. 11 daha büyük olduğu için 11, 7'nin yerine kaydırılır. Önünde başka eleman yoktur; 7 boşalan yere yazılır. Dizi: $7, 11, 15, 3, 16, 13$.
- **Anahtar 15:** 15, 11 ile karşılaştırılır; 15 daha büyüktür, hiçbir yer değişikliği olmaz. Dizi aynı kalır.
- **Anahtar 3:** 15, 3'ten büyüktür; 3'ün yerine kaydırılır. 11 de 3'ten büyüktür; 15'in yerine kaydırılır. 7 de 3'ten büyüktür; 11'in yerine kaydırılır. 7'nin gözü boş kalır; 3 oraya yerleşir. Dizi: $3, 7, 11, 15, 16, 13$.
- **Anahtar 16:** Önündeki elemanların hepsinden büyüktür; yapılacak bir şey yoktur.
- **Anahtar 13:** 16, 13'ten büyüktür; kaydırılır. 15 de büyüktür; kaydırılır. 11 ile karşılaştırılınca anahtar daha büyüktür; 13, 15'in boşalan yerine yerleşir. Dizi: $3, 7, 11, 13, 15, 16$.

Adım	Anahtar	Dizi
1	7	7, 11, 15, 3, 16, 13
2	15	7, 11, 15, 3, 16, 13
3	3	3, 7, 11, 15, 16, 13
4	16	3, 7, 11, 15, 16, 13
5	13	3, 7, 11, 13, 15, 16

Yürütme zamanı: Bu yinelemeli bir algoritmadır ve görülen yöntemlerle rahatça analiz edilebilir.

- j'ye 2 atanır ve döngü $j = 2$'den n'e kadar gider. Son sınırın kontrolü ve j'nin artırılması n civarında yapılır.
- Döngü içinde `key = A[j]` ataması ve `i = j - 1` ataması vardır; ikisi de $n - 1$ kez yapılır.
- İçeride bir `while` vardır; iki karşılaştırması bulunur ($i > 0$ ve `A[i] > key`). Bu `while` döngüsünün kaç kez döneceği j'ye bağlıdır. j büyüdükçe daha fazla döner. Bu yüzden, 1'den n'e kadar sayıların toplamında olduğu gibi bir hesap yapılır. `while` içindeki kaydırma ataması ve i'nin azaltılması da o kadar yapılır.
- `while` dışındaki yerleştirme ataması $n - 1$ kez yapılır.

Slayttaki tablo bir kitaptan alınmıştır ve `while` döngüsünün j için tekrar sayısı t_j ile gösterilmiştir. Hoca, kendi yaptığı gibi "1'den n'e kadar sayıların toplamı $\frac{n(n+1)}{2}$" diye açılabileceğini ya da t_j'ler cinsinden bırakılabileceğini söyledi. `while` içindeki satırların bir adım eksik çalışmasının nedeni, koşul sağlanmadığı anda gövdeye girilmemesidir.

Karmaşıklık: Terimlerde n'ler ve n^2'ler vardır. Maksimum olan alınır: n^2. Karmaşıklık tablolarına bakıldığında da araya ekleme sıralamasının en kötü durumda $O(n^2)$, ortalama durumda $O(n^2)$ ve en iyi durumda $O(n)$ olduğu görülür.

> **Ders sonrası not:** Hoca `while` döngüsünün tekrarını "$j = 2$ iken bir kere, $j = 3$ iken iki kere, $j = 4$ iken dört kere" diye saydı. Doğru sayım $j - 1$'dir: $j = 4$ için en kötü durumda üç kez döner. Toplam $1 + 2 + \cdots + (n - 1) = \frac{n(n-1)}{2}$ olur. Hoca en iyi durumu da "ilk birinci de bulmuş" diye anlattı. Doğrusu: dizi zaten sıralıysa her eleman tek karşılaştırmayla yerinde kalır; toplam $n - 1$ karşılaştırma, $O(n)$.

24.2 Seçmeli Sıralama

Seçmeli sıralama (selection sort), genelde ilk öğretilen iki döngülü sıralamadır. Hoca bunu kendi başınıza çalışmanız için kullanmanızı, kendisinin yaptığı gibi $T(n)$'i çıkarıp karmaşıklığı bulmanızı ve sonra tablolardaki en kötü durumla karşılaştırmanızı önerdi.

Slayttaki örnekte dizi $17, 33, 14, 27, 18$ gibi elemanlardan oluşur. İlk turda 17, 33 ile karşılaştırılır; 33 daha büyüktür. 17, 14 ile karşılaştırılır; 14 daha küçük olduğu için en küçük aday 14 olur. 14, 27 ve 18 ile karşılaştırılır; değişiklik olmaz. Tur sonunda en küçük eleman olan 14 başa gelir. Sonraki turda 17 ile başlanır ve kalan elemanlarla karşılaştırılır; böyle devam edilir.

Analiz: Hoca bu slaytta $T(n)$ çıkarmaktan vazgeçtiğini, yalnız döngülere baktığını söyledi.

- Dış döngü 0'dan $n - 1$'e kadar döner; $n - 1$ kez.
- İç döngü $j + 1$'den başlar. Dış indis 0 iken iç döngü $n - 1$ kez, 1 iken $n - 2$ kez, 2 iken $n - 3$ kez döner; her turda bir azalır.
- Toplam karşılaştırma sayısı:

$$(n - 1) + (n - 2) + \cdots + 1 = \frac{n(n - 1)}{2}$$

$n-1$ ile bu ifade çarpıldığında en büyük kuvvet n^2 olur; diğer terimler n'e bağlıdır. Karmaşıklık $O(n^2)$'dir. Seçmeli sıralamada en iyi, en kötü ve ortalama durumun hepsi $O(n^2)$'dir.

25. Dersin Kapanışı

Son konu olarak özyinelemeli (recursive) algoritmaların analizine geçilecekti. Hoca bu konuyu aceleye getirmek istemediği, daha iyi anlatmak istediği için bu bölüme geçmedi.

Kapanışta hoca şu tavsiyeleri verdi:

- Algoritmik düşünmeyi iyi öğrenin. İkinci sınıfı bitirdiğinizde algoritmayla ilgili bir probleminiz kalmamalı.
- Kod yazın. Derste anlatılan algoritmaların kodunu kendiniz yazın; önce o algoritmayı yazanın nasıl düşündüğünü analiz edin, sonra "ben nasıl yaparım" diye düşünün.
- Temelinizi iyi kurun; ondan sonrası çok daha farklı olur.

- Anlamadığınız bir yer olursa arkadaşınıza sorun, internetten araştırın, her şeyi kendiniz üzerine koyun.

  Hocanın vurgusu: Hep kendinle yarış, başkasıyla yarışma. Başkasıyla yarışmak insanı raydan çıkarıyor.

Hoca, slaytlarda ve anlatımda eksik ve hata olabileceğini, görülürse söylenmesini bir kez daha istedi.

Örnek Sorular

Bu derste hazır kare yoktur. Aşağıdaki sorular derste sözlü olarak sorulan ya da sınıfla birlikte çözülen sorulardır.

Soru 1 (derste sözlü, 00:36:52)

$A * B/C + (C/D - A)$ ifadesini prefix ve postfix gösterimde yazınız.

> **Çözüm:** İşlem sırası: $A * B$, sonuç $/C$; C/D, sonuçtan A çıkar; iki sonucu topla. Prefix: $+\ /\ *\ A\ B\ C\ \ -\ /C\ D\ A$. Postfix: $A\ B\ *\ C\ /\ C\ D\ /A\ -\ \ +$ (Bölüm 6.2).

Soru 2 (derste sözlü, 00:41:28)

$\dfrac{b^2 - 4ac}{2a}$ ifadesini ikili ağaca yerleştirip prefix ve postfix gösterimde yazınız.

> **Çözüm:** Kökte $/$, solda $b^2 - 4ac$, sağda $2a$. Prefix: $/\ -\ \uparrow\ b\ 2\ \ *\ *4\ a\ c\ \ *\ 2\ a$. Postfix: $b\ 2\ \uparrow\ \ 4\ a\ *\ c\ *\ \ -\ 2\ a\ *\ \ /$ (Bölüm 6.3).

Soru 3 (derste sözlü, 00:47:42)

$(3 + 2)^2 + 8/4 * 3$ ifadesinin prefix ve postfix gösterimini yazınız. Sonucu nedir?

> **Çözüm:** Prefix: $+\ \uparrow\ +3\ 2\ 2\ \ *\ /8\ 4\ 3$. Postfix: $3\ 2\ +\ 2\ \uparrow\ \ 8\ 4/3\ *\ \ +$. Değer: $5^2 = 25$, $8/4 = 2$, $2 \cdot 3 = 6$, $25 + 6 = 31$ (Bölüm 7).

Soru 4 (derste sözlü, 01:58:32 ve 01:59:07)

Aynı problemi çözen iki algoritmanın yürütme zamanları $T_A(n) = 50n$ ve $T_B(n) = n^2$'dir. Hangisini seçersiniz?

> **Çözüm:** $n < 50$ için B daha hızlıdır, ama $n > 50$ için B'nin süresi hızla artar. Girişin 50'yi geçmeyeceği garanti değilse A seçilir (Bölüm 17).

Soru 5 (derste sözlü, 02:16:16)

Sıralı aramanın en iyi, en kötü ve ortalama durumu nedir?

> **Çözüm:** En iyi 1, en kötü n karşılaştırma. Ortalama: $\dfrac{1 + 2 + \cdots + n}{n} = \dfrac{n+1}{2}$. Karmaşıklık $O(n)$ (Bölüm 20.1).

Soru 6 (derste sözlü, 02:18:20)

Sıralı arama algoritmasının $T(n)$'ini hesaplayınız.

Soru 7 (derste sözlü, 02:30:13)

Bir algoritmanın iki parçasından birinin karmaşıklığı $O(n)$, diğerininki $O(\log n)$'dir. Algoritmanın karmaşıklığı nedir? Parçalar $O(2^n)$ ve $O(n)$ olsaydı?

Çözüm: Toplanan parçalarda en büyüğü alınır. $O(n) + O(\log n) = O(n)$; $O(2^n) + O(n) = O(2^n)$ (Bölüm 23.1).

Soru 8 (derste sözlü, 02:31:09)

$f(n) = 3n \log(n!) + (n^2 + 3) \log n$ fonksiyonunun Big-O'su nedir?

Çözüm: $3n = O(n)$, $\log(n!) = O(n \log n)$; çarpım $O(n^2 \log n)$. $(n^2 + 3) \log n = O(n^2 \log n)$. Toplam: $O(n^2 \log n)$ (Bölüm 23.2).

Soru 9 (derste sözlü, 02:36:48)

$f(x) = (x + 1) \log(x^2 + 1) + 3x^2$ fonksiyonunun Big-O'su nedir?

Çözüm: $(x + 1) = O(x)$, $\log(x^2 + 1) = O(\log x)$; çarpım $O(x \log x)$. $3x^2 = O(x^2)$. Toplam: $O(x^2)$ (Bölüm 23.3).

Soru 10 (derste sözlü, 02:24:57)

$n > 6$ olan tam sayılar için $3^n < n!$ olduğunu tümevarımla ispatlayınız.

Çözüm: Taban: $3^7 = 2187 < 5040 = 7!$. Adım: $3^k < k!$ ise $3^{k+1} = 3 \cdot 3^k < 3 \cdot k! < (k + 1) \cdot k! = (k + 1)!$. Buradan $3^n = O(n!)$ (Bölüm 22.2).

Dersin Toparlaması

Bu derste algoritmalar bölümüne temel veri yapılarıyla devam edildi. Dizi, bağlı liste, kuyruk (FIFO) ve yığın (LIFO) tanıtıldı; bağlı listeye eleman eklemenin yalnız linkleri değiştirdiği, yığının push ve pop işlemleriyle ve yığın göstergesiyle çalıştığı görüldü. Yığının önemli bir uygulaması olarak infix, prefix ve postfix gösterimleri işlendi. İfadeler ikili ağaca yerleştirilip prefix ve postfix biçimlere çevrildi. Infix ifadenin iki yığınla, prefix ve postfix ifadelerin tek yığınla değerlendirildiği; bu yüzden derleyicilerin ifadeleri prefix ya da postfix biçime çevirdiği anlatıldı.

İkinci bölümde algoritma analizine geçildi. Temel işlemler sayılarak yürütme zamanı $T(n)$ hesaplandı: ortalama bulma $4n + 6$, en küçük elemanı bulma $4n$, seçmeli sıralama $2n^2 + 9n + 2$, matris toplama $4n^2 + 4n + 2$. Fonksiyonların büyüme sırası, en kötü, en iyi ve ortalama durum, Big-O, Big-Omega ve Big-Theta tanımları ve tanık sabitleri görüldü. Karmaşıklığın limit ve tümevarımla gösterilmesi, parçalı fonksiyonlarda toplam ve çarpım kuralları ve araya ekleme ile seçmeli sıralamanın analizi yapıldı. Özyinelemeli algoritmaların analizi ve rekürans bağıntıları sonraki derslere bırakıldı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.